



Assignment 2 (Graded)

Technical Assignment: Backend Server Development with Node.js, Express, and MongoDB

Overview

In this assignment, you will build a robust backend using **Node.js**, **Express**, and the **mongodb** package. You will design a schema-like structure for user profiles using 10 distinct data types, implement endpoints to receive and retrieve data, and build filtering capabilities for data retrieval.

Phase 1: Project Plan & Architecture

Before writing code, you must document your architecture. Your project plan should include:

- **API Directory Structure:** A clean layout (e.g., separating server configuration and database connection).
- **Endpoint Registry:** A list of your 4 endpoints with their corresponding HTTP methods, URI paths, and expected request/response structures.
- **Database Schema Mapping:** A breakdown of how your user profile object maps to 10 distinct JSON/JavaScript data types.

Phase 2: Technical Requirements

1. Database Configuration

- You must use MongoDB package for Mongo Atlas connection and interactions.
- Implement a connection module that establishes a connection to your MongoDB instance online and reuses the database client efficiently across your application.

2. Data Structure Specification (User Profiles), Similar for pet information

Each user document stored in your MongoDB collection must utilize at least **10 different data types**. Use the following blueprint for your document structure:

Field Name Example Value

<code>_id</code>	<code>ObjectId("60c72b2f9b1d8b2bad754f21")</code>
<code>fullName</code>	<code>"Jane Doe"</code>
<code>age</code>	<code>28</code>
<code>hourlyRate</code>	<code>45.50</code>
<code>isActive</code>	<code>true</code>
<code>joinDate</code>	<code>2026-05-15T12:00:00Z</code>
<code>skills</code>	<code>["JavaScript", "Node.js", "Express"]</code>
<code>address</code>	<code>{ "city": "Toronto", "country": "Canada" }</code>

3. API Endpoint Requirements

POST Requests (Data Ingestion) example -

- **POST /api/users:** Accepts a single user profile object matching the data specification, processes the inputs, and inserts it into the MongoDB collection.
- **POST /api/pets:** Accepts a single pet related object matching the data specification, processes the inputs, and inserts it into the MongoDB collection.

GET Requests (Data Retrieval & Filtering) example -

- **GET /api/users:** Returns a list of all users from the database.
- **GET /api/users/search:** Implements **query parameter filtering**.
The endpoint must parse query strings to filter the MongoDB collection dynamically.
 - *Example:* GET /api/users/search?city=Toronto&age=25 should return only active users living in Toronto who are 25 years old.
 - *Implementation Tip:* Build the MongoDB query object dynamically based on req.query.
- **GET /api/pets:** Returns a list of all pets from the database.

Due date - Submit before the class on Tuesday



100 % 1 of 1 topics complete

Assignment 2 (Graded)

Assignment





Project

Initialize Project & Install Dependencies

Open your terminal, create a new directory for your project, initialize it, and install the native MongoDB driver, Express, and Dotenv (to manage your connection strings safely).

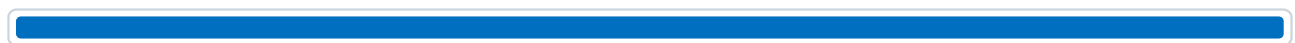
Create the Database Connection Module

Using the native mongodb package requires managing the client connection manually. Create function to reuse that same database instance across your app.

Create the Main App File (server.js)

Combine everything in your entry point. This file sets up Express middleware, points to the routing rules, calls the database initializer, and sets the app to listen on your configured port.

Share the app.js or the main file in your project, Package.json and project related files and folder. Do not send node_module or package-lock.json



100 % 1 of 1 topics complete

final project

Assignment





Project

Project Title

User & Pet Management System (Angular 22)

Objective

Build an Angular frontend that communicates with the Node.js backend created in the previous assignment.

The application must:

- Navigate using Angular Routes
- Display different pages
- Use standalone reusable components
- Submit data to backend APIs
- Display data retrieved from MongoDB
- Reuse components across multiple pages

Required Routes

Route	Component
/	Home Page
/add-user	Add User Page
/add-pet	Add Pet Page
/users	Display Users
/pets	Display Pets

Home Page

Purpose:

Navigation page of the application.

Must contain

- Application Title
- Welcome message
- Buttons or links

Add User Page

This page only acts as a container.

It should contain

<Page Title Component>

<User Form Component>

The actual form must be inside a standalone component.

User Form Component

This reusable component contains the form.

It submits data to

POST /api/users

Form Fields

Field	Type
Full Name	Text
Age	Number
Hourly Rate	Number
Is Active	Checkbox
Join Date	Date

Field	Type
Skills	Text (comma separated)
City	Text
Country	Text

Add Pet Page

Container page only.

Contains

<Page Title>

<Pet Form Component>

Pet Form Component

Calls

POST /api/pets

Suggested Fields

Field	Type
Pet Name	Text
Animal Type	Text
Breed	Text
Age	Number
Weight	Number
Vaccinated	Checkbox
Owner Name	Text
Color	Text

Users Page

Container page.

Contains

<Page Title>

<User List Component>

Pets Page

Contains

<Page Title>

<Pet List Component>

Shared Components

1. Navbar Component

Appears on every page.

Contains links

Home

Add User

Add Pet

Users

Pets

2. Page Title Component

Reusable component

Input

title

Angular Concepts Students Must Use

- Standalone Components
- Angular Routing
- Router Outlet
- Page Components
- Reusable Components
- Component Inputs
- Event Binding
- Two-way Binding ([(ngModel)])
- FormsModule
- HttpClient
- Lifecycle Hook (ngOnInit)
- @for (or *ngFor)
- JSON Objects

****Submit the project as zip and remove node_modules from it.**

****Project to be submitted before the class on monday July 13th**

0 % 0 of 1 topics complete

MEAN Stack final project



Assignment